Functor wrapper
Data.Functor.Const : https://hackage.haskell.org/package/base/docs/Data-Functor-Const.html
Data.Functor.Identity : https://hackage.haskell.org/package/base/docs/Data-Functor-Identity.html
Data.Functor.Sum : https://hackage.haskell.org/package/base/docs/Data-Functor-Sum.html
Data.Functor.Product : https://hackage.haskell.org/package/base/docs/Data-Functor-Product.html
Data.Functor.Compose : https://hackage.haskell.org/package/base/docs/Data-Functor-Compose.html

```
-- Data.Functor.Identity
newtype Identity a = Identity { getIdentity :: a -> a }
instance Functor Identity
instance Applicative Identity
instance Monad Identity

-- Data.Functor.Const
newtype Const a b = Const { getConst :: a }
instance Functor (Const a)
instance Monoid a => Applicative (Const a)

-- Data.Functor.Compose
newtype Compose f g a = Compose { getCompose :: f (g a) }
instance (Functor f, Functor g) => Functor (Compose f g)
instance (Applicative f, Applicative g) => Applicative (Compose f
g)

-- Data.Functor.Sum
newtype Sum f g a = InL (f a) | InR (g a)
instance (Functor f, Functor g) => Functor (Sum f g)
instance (Contravariant f, Contravariant g) => Contravariant (Sum f
g)

-- Data.Functor.Product
newtype Product f g a = Pair (f a) (g a)
instance (Functor f, Functor g) => Functor (Product f g)
instance (Applicative f, Applicative g) => Applicative (Product f
g)
instance (Monad f, Monad g) => Monad (Product f g)
instance (Contravariant f, Contravariant g) => Contravariant
(Product f g)
```

Functors wrapper
Common **abstract type** as a functor

Identity a	value as a functor
f <$> Identity a = Identity (f x)	functor
Identity f <*> Identity x = Identity (f x)	applicative
Identity x >>= k = k x	monad

```
  Const a                                          constant as a functor
  f <$> Const a = Const a                          functor       i.e. ignores map
  Const x <> Const y = Const (x <> y)              monoid
  Const x <*> Const y = Const (x <> y)             applicative  i.e. ignores map

  Compose f g a                                    composition type (f (g a)) as a
functor
  f <$> Compose x = Compose ((f <$>) <$> x)        functor on inner type            (a ->
b) → f (g a) → f (g b)
  Compose f <*> Compose x                          applicative  on inner type       f (g (a
-> b)) → f (g a) → f (g b)
      = Compose ((f <*>) <*> x)


  Sum f g a                                        algebraic sum type (f a) | (g a) as a
functor
  f <$> InL x = InL (f <$> x)                      functor     (a -> b) →  (f a) + (g a)
→ (f b) + (g b)
  f <$> InR x = InR (f <$> x)

  Product f g a                                    algebraic product type (f a) x (g a)
as a functor
  f <$> Pair x y = Pair (f <$> x) (f <$> y)        term-wise functor            (a -> b)
→  (f a) x (g a) → (f b) x (g b)
  Pair f g <*> Pair x y = Pair (f x) (g x)         term-wise applicative        (f (a-> b))
x (g (a -> b)) →  (f a) x (g a) → (f b) x (g b)
  Pair x y >>= k                                   term-wise monad              (f a) x (g
a) → (a -> (f b) x (g b)) → (f b) x (g b)
    = Pair (x >>= fst.r.k) (y >>= snd.r.k)
   where r (Pair x y) = (x, y)
```

```
import Data.Functor.Identity
import Data.Functor.Const
import Data.Functor.Sum
import Data.Functor.Product
import Data.Functor.Compose

-- Identity a
-- i.e. raise any value to a trivial functor
Identity :: a -> Identity a
x = Identity 1
runIdentity x


---- Compose f g a
-- i.e. nested functors
Compose :: f (g a) -> Compose f g a
x = Compose [Just 'a']    -- Compose [] Maybe Char     i.e.
composition of list+Maybe
getCompose x
x = Compose (Just 'a')    -- ERROR. Not nested
x = Compose 'a            -- ERROR.


---- Sum f g a
-- i.e. algebraic sum of functors
```

```
InL :: f a -> Sum f g a
InR :: g a -> Sum f g a
x = InL (Just 'a')     -- InL (Maybe Char)
x = InR (Just 'a')     -- InR (Maybe Char)
x = InL 'a'            -- ERROR. Not a functor


---- Product f g a
-- i.e. cartesian product of functors
Pair :: f a -> g a -> Product f g a
x = Pair (Just 'a') (Right 'a')    -- Product Maybe Either Char
x = Pair (Just 'a') (Right 1)      -- ERROR. Inner type must be
same.
x = Pair (Just 'a') 1              -- ERROR


---- Const a b
-- i.e. same as const a b
Const :: a -> Const a b
x = Const 1
getConst x


-- Map semantics
-- i.e. how each value is mapped
-- In general, behaves as expected
(*2) <$> (Identity 1)                -- Identity 2
(*2) <$> (Const 1)                   -- Const 2
(*2) <$> InL (Just 1)                -- InL (Just 2)
(*2) <$> InR (Just 1)                -- InR (Just 2)
(*2) <$> (Pair (Just 1) (Right 2))   -- fmap on both. Pair (Just 2)
(Right 4)
(*2) <$> Compose (Right (Just 1))    -- fmap on inner. Compose
(Right (Just 2))
(*2) <$> Compose (Left (Just 1))     -- unchanged i.e. (Left (Just
1)) since Left is not mapped
(*2) <$> Compose (Right Nothing)     -- unchanged i.e. (Right
Nothing)
```

Contravariant wrappers
Ref: https://hackage.haskell.org/package/base/docs/Data-Functor-Contravariant.html

```
newtype Op a b = Op { getOp :: b -> a }
instance Contravariant (Op a)
instance Monoid a => Monoid (Op a b)
instance Semigroup a => Semigroup (Op a b)

newtype Predicate a = Predicate { getPredicate :: a -> Bool }
instance Contravariant Predicate
instance Monoid a => Monoid (Predicate a)
instance Semigroup a => Semigroup (Predicate a)
```

```
newtype Comparison a = Comparison { getComparison :: a -> a ->
Ordering }
newtype Equivalent a = Equivalent { getEquivalence :: a -> a ->
Bool }
instance Contravariant Comparison
instance Monoid a => Monoid (Comparison a)
instance Semigroup a => Semigroup (Comparison a)
instance Contravariant Equivalent
instance Monoid a => Monoid (Equivalent a)
instance Semigroup a => Semigroup (Equivalent a)
```